

项目提升 workflow 安排与分工

一、改进全景图

维度	当前状态	目标状态
Agent 架构	LangChain ReAct (recursion_limit=8)	LangGraph 状态机（显式节点 + 自省环路）
LLM 接口	Prompt → JSON 字符串解析	DeepSeek Function Calling 结构化工具调用
实验完整性	28条，无消融，无Baseline	60+ 条 + 消融矩阵 + Baseline 对比表
前端	Streamlit 单页	React/Next.js · Claude 风 对话界面
报告展示	终端文字 / JSON	三类报告 PDF 内嵌 + 电路图渲染
知识库	29条，弱国产覆盖	50+ 条，含矽力杰/圣邦微专项
检索	纯向量检索	混合检索（BM25 + 向量 + RRF）
缓存	无	语义缓存层（相似度 > 0.95 命中）
输出体验	批量返回（~4.5s 白屏）	SSE 流式逐段输出
电路图	无	schemdraw 参数化应用电路 SVG
导出	无	BOM 清单 Excel 导出
风险可视化	文字标签	5×5 风险热力矩阵

二、技术选型决策

前端（全量重构）

层次	选型	原因
框架	Next.js 14 (App Router) + TypeScript	SSR、文件路由、API Routes, 工程完整度高
样式	Tailwind CSS + shadcn/ui	极简设计系统, Claude/GPT 同款审美基调
动效	Framer Motion	消息气泡入场、打字机效果、面板切换
状态管理	Zustand	轻量, 管理多 session 会话历史
流式接收	EventSource (SSE)	对接 FastAPI StreamingResponse, 实时打字输出
PDF 展示	@react-pdf-viewer/core (PDF.js 封装)	三类报告在对话中内嵌渲染, 无需下载
电路图	内嵌 SVG (后端 schemdraw 生成)	参数化应用电路, 随选型结果实时更新
图表	Recharts	风险热力图、评分雷达图

后端（增量升级）

模块	选型	说明
Agent 框架	LangGraph 0.2+	替换 LangChain ReAct, 显式有向图
LLM 接口	DeepSeek Function Calling API	绑定 Pydantic schema, 弃用字符串 JSON 解析
流式输出	FastAPI StreamingResponse + SSE	<code>/analyze/stream</code> 新端点
电路图生成	schemdraw 0.18	参数化 Buck/Boost/LDO 应用电路 SVG
检索	BM25 (rank_bm25) + ChromaDB + RRF 融合	混合检索
语义缓存	本地 ChromaDB 缓存集合	cosine > 0.95 → 跳过 LLM 调用
Excel 导出	openpyxl	BOM 清单一键导出

三、阶段计划（建议顺序）

Day 1-2	└─ [基础重构] LangGraph + Function Calling + SSE 接口
Day 2-4	└─ [前端主体] Next.js 项目初始化 + 对话界面 + SSE 接入
Day 3-5	└─ [电路图] schemdraw 生成服务 + 前端 SVG 嵌入
Day 3-5	└─ [实验补全] 消融实验 + Baseline 对比 + 60+ 测试集
Day 5-6	└─ [功能增强] 混合检索 + 语义缓存 + 自省节点 + 知识库扩展
Day 6-7	└─ [集成收尾] PDF 内嵌 + BOM 导出 + 风险热力图 + 联调测试

四、三人详细分工

队长（负责 Agent 核心 + 实验 + 集成）

P1: LangGraph 架构重构

目标：将 `app/agent_orchestrator.py` 和 `app/react_agent.py` 迁移至 LangGraph 状态机

任务清单：

☐ 设计状态图节点：

```
ParseNode → SearchNode → ScoreNode → EvidenceNode → CriticNode →
ReportNode
```

↑ |
└─ 不一致则回退 ┘

☐ 实现 `ParseNode`：调用 DeepSeek Function Calling（见 P2），输出

`RequirementConstraints`

☐ 实现 `CriticNode`（自省节点）：

- 输入：已生成的 `TopologyIR` + 原始 `RequirementConstraints`
- 检查项：拓扑与电压方向是否一致、推荐器件参数是否满足需求
- 不通过：修改 `state["retry"] = True`，回退到 `SearchNode`
- 上限：最多重试 1 次，避免死循环

☐ 添加条件边：`ScoreNode` 后，若候选数 < 3 则自动放宽电流下限 20% 重搜

☐ 迁移 `Multi-turn ReAct`：`react_agent.py` 保留，改为调用 LangGraph 图对象

☐ 验证：28 条测试用例全部通过，延迟中位数不超过 6s

参考文件：`app/agent_orchestrator.py`、`app/react_agent.py`、`app/schemas.py`

P2: DeepSeek Function Calling 结构化工具调用

目标：替换 `llm_client.py` 中 `parse_requirement_with_llm()` 的 JSON 字符串解析模式

任务清单：

☐ 定义 Tool Schema（对应 RequirementConstraints 的每个字段）：

```
tools = [{
    "type": "function",
    "function": {
        "name": "extract_requirement",
        "parameters": {
            "type": "object",
            "properties": {
                "category": {"type": "string", "enum":
["dc_dc_converter", "ldo", ...]},
                "topology": {"type": "string", "enum": ["buck", "boost",
"buck_boost", "ldo"]},
                "input_voltage_nominal_v": {"type": "number"},
                # ... 其余字段
            }
        }
    }
}]
```

- ☐ 修改 API 调用方式：tool_choice="required" 强制调用，从 tool_calls[0].function.arguments 取结果
- ☐ L2/L3/L4 容错链保留：Function Calling 提升 L1 可靠性，L2-L4 仍作为后置校验
- ☐ 单元测试：新增 5 条故意构造的歧义输入，验证 FC 模式下解析稳定性

P3：消融实验 + Baseline 对比

目标：为四级容错链和双模评分提供实验证据，直接回应评委追问

任务清单：

☐ 消融实验设计：

实验组	关闭哪一级	跑哪些用例	记录指标
Full（基准）	无	全部 28 条	7 字段准确率
w/o L2	规则覆盖层	全部 28 条	7 字段准确率
w/o L3	电压方向校验	全部 28 条	7 字段准确率
w/o L4	温度兜底	全部 28 条	7 字段准确率
w/o L2+L3+L4	仅 LLM 初解	全部 28 条	7 字段准确率

- ☐ 在 eval_runner.py 中添加 --ablation 参数，控制关闭哪一级
- ☐ Baseline 对比设计：
 - 用裸 DeepSeek-V3（无四级链、无规则引擎）对同样 28 条输入提问
 - 记录字段准确率、幻觉率（型号错误 / 拓扑错误）

- 生成两列对比表：Baseline VS 本系统

☐ 输出：在 docs/eval_results_ablation.md 中保存实验数据表格

P4: 扩展测试集 (28 → 60+ 条)

任务清单：

- ☐ 新增边缘用例覆盖：
 - 极小电流 (50mA LDO)
 - 输入输出电压相近边界 (12V→11V)
 - 纯口语描述 ("帮我找个五伏转三伏三的芯片，电流一安培左右")
 - 混合单位 ("input 24V, output 500mA at 3.3V")
 - 缺省字段 (无温度范围、无等级要求)
 - 国产偏好 ("优先矽力杰或圣邦微")
 - 车规高温 (-40~150°C AEC-Q100 Grade 0)
- ☐ 同步更新 tests/test_cases_60.xlsx
- ☐ 在新测试集上重跑消融实验，汇总整体结论

P5: 自省节点 (CriticNode)

已在 P1 LangGraph 重构中包含，此处单独列出实现要点

☐ Critic Prompt 设计：

你是一位挑剔的硬件工程师。请审核以下选型结论是否存在明显错误：

- 拓扑声称 {topology}，但 Vin={Vin}V, Vout={Vout}V，方向是否一致？
- 推荐器件 {mpn}，额定电流 {Irated}A，需求电流 {Irequired}A，是否满足？
- 有无明显的参数矛盾？

若有问题，输出 JSON {"valid": false, "reason": "..."}；否则 {"valid": true}

- ☐ 触发条件：仅在 LLM 增强模式下运行（纯规则模式跳过）
- ☐ 记录：在报告中注明是否经过自省校验

队友 A（负责前端全量重构）

F1: Next.js 项目初始化与设计系统

目标：在 frontend/ 目录下搭建完整 Next.js 应用，建立设计语言

任务清单：

- ☐ 初始化项目：

```
cd frontend
npx create-next-app@latest web --typescript --tailwind --app
cd web
npx shadcn-ui@latest init
```

☐ 安装依赖:

```
npm install framer-motion zustand @react-pdf-viewer/core @react-pdf-viewer/default-layout recharts react-markdown remark-gfm
npm install -D @types/node
```

☐ 设计 Token 定义 (tailwind.config.ts) :

- 主色: #1a237e (品牌蓝)
- 背景: #f8f9fa (浅灰)
- 卡片: #ffffff
- 风险色: HIGH #c62828、Medium #f57c00、Low #2e7d32

☐ 全局布局: 三栏 layout (左: 会话历史侧边栏 260px, 中: 主对话区, 右: 详情面板 400px)

☐ 暗色模式: 基于 next-themes, 跟随系统

F2: Claude 风对话界面

目标: 主对话区实现仿 Claude 的消息气泡布局 + 流式打字效果

任务清单:

☐ 消息组件 (components/MessageBubble.tsx) :

- 用户消息: 右对齐, 蓝色气泡, 圆角
- 系统消息: 左对齐, 白色卡片, 头像区 (eZ-PLM logo)
- 支持 Markdown 渲染 (react-markdown + remark-gfm)
- 代码块高亮 (prism-react-renderer)

☐ 流式打字效果:

- 对接 SSE (EventSource 连接后端 /analyze/stream)
- 逐 token 追加到消息尾部, 光标闪烁动效 (CSS animate-pulse)
- 完成后显示"已完成 · 耗时 4.3s"角标

☐ 输入区: 多行 textarea, Enter 发送 (Shift+Enter 换行), 附带发送按钮

☐ 会话历史侧边栏:

- Zustand store 管理 sessions[]
 - 每条会话显示: 首条输入前 30 字 + 时间戳
 - 点击切换历史会话, 支持删除
-

F3: 三类报告内嵌展示

目标：在对话中以卡片 + Tab 形式直接展示三类报告，无需跳转

三类报告：

1. 选型报告 (`SelectionReport`) — 推荐器件列表 + 参数对照
2. 风险评估报告 (`RiskIR`) — 风险列表 + 风险热力矩阵
3. 替换建议报告 (`ReplacementReport`) — 替代型号对比

任务清单：

- ☐ 报告卡片组件 (`components/ReportCard.tsx`) :
 - Tab 切换三类报告
 - 选型报告：表格展示 PartIR，评分用进度条，Badge 显示等级
 - 风险报告：HIGH/MEDIUM/LOW 分色列表 + 置信度 chip
 - 替换报告：原型号 vs 替代型号参数差异高亮对比
 - ☐ PDF 内嵌 (`components/PdfViewer.tsx`) :
 - `@react-pdf-viewer/core` 渲染后端返回的 PDF 字节流
 - 支持缩放、全屏、下载
 - 报告卡片底部设"查看完整 PDF"按钮展开
 - ☐ 报告触发逻辑：
 - 收到 SSE 完成信号后，向 `/api/report/pdf` 请求 PDF
 - 同时渲染结构化卡片 (JSON → React 组件)
-

F4: 电路图展示组件

目标：在选型结果下方展示推荐器件的参数化应用电路图 (SVG)

任务清单：

- ☐ 电路图面板 (`components/SchematicPanel.tsx`) :
 - 向后端 `/schematic/{topology}?Vin=12&Vout=5&Iout=3` 发起请求
 - 后端返回 SVG 字符串，内嵌 `<div dangerouslySetInnerHTML>` 渲染
 - 支持缩放 (`transform-origin`) 和全屏展示 (Framer Motion `AnimatePresence`)
 - ☐ 标注层 (Overlay) :
 - SVG 元件旁叠加计算值标注 (电感值、电容值、分压电阻)
 - 鼠标悬停显示元件参数 Tooltip
 - ☐ 拓扑说明卡：电路图下方显示拓扑类型 (Buck/Boost/LDO) + 关键参数方程
-

F5: 风险热力矩阵 + BOM 导出入口

- ☐ 风险热力矩阵 (`components/RiskHeatmap.tsx`) :

- 5×5 网格（横轴：严重度 1-5，纵轴：发生概率 1-5）
- 每个候选器件用散点表示，颜色编码 HIGH/MEDIUM/LOW
- 悬停显示器件型号 + 主要风险描述
- 使用 Recharts `ScatterChart`

☐ BOM 导出按钮：

- 调用后端 `/export/bom` 接口
- 浏览器自动下载 `.xlsx` 文件
- 按钮设计：带 Excel 图标，悬停显示"导出 BOM 清单 (.xlsx)"

队友 B（负责后端增量升级）

B1: FastAPI SSE 流式输出接口

目标：新增 `/analyze/stream` 端点，支持逐段 SSE 推送报告内容

任务清单：

☐ 新增端点（`app/main.py`）：

```
@app.post("/analyze/stream")
async def analyze_stream(req: AnalyzeRequest):
    return StreamingResponse(
        _stream_analyze(req),
        media_type="text/event-stream",
        headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"}
    )
```

☐ 实现 `_stream_analyze()` 生成器，按阶段推送 SSE 事件：

- `event: parse_done` — 解析完成
- `event: search_done` — 搜索完成 + 候选数量
- `event: score_update` — 每个器件评分完成 + PartIR
- `event: risk_done` — 风险评估完成 + RiskIR
- `event: text_delta` — 报告文本逐 token
- `event: done` — 完成 + 总耗时

☐ CORS 配置：允许 `localhost:3000`（Next.js dev server）

☐ 测试：用 curl 验证 SSE 正常推送，无阻塞

B2: schemdraw 参数化电路图生成

目标：根据拓扑 + 设计规格，生成带计算值的应用电路 SVG

任务清单：

☐ 安装依赖：`pip install schemdraw`

☐ 新增 `app/schematic_generator.py` :

- `generate_buck_schematic(Vin, Vout, Iout, fsw=500e3)` → SVG str
 - 计算 L: $L = Vin \times D \times (1-D) / (fsw \times \Delta IL)$, 其中 $D = Vout/Vin$, $\Delta IL = 0.3 \times Iout$
 - 计算 Cout: $Cout = \Delta IL / (8 \times fsw \times \Delta Vout)$, $\Delta Vout = 0.01 \times Vout$
 - schemdraw 绘制完整 Buck 拓扑 (SW管、续流二极管、电感、输入输出电容)
 - 元件旁标注计算值
- `generate_boost_schematic(Vin, Vout, Iout)` → SVG str
- `generate_ldo_schematic(Vin, Vout, Iout)` → SVG str

☐ 新增端点 (`app/main.py`) :

```
@app.get("/schematic/{topology}")
async def get_schematic(topology: str, Vin: float, Vout: float, Iout: float):
    svg = generate_schematic(topology, Vin, Vout, Iout)
    return Response(content=svg, media_type="image/svg+xml")
```

☐ 测试: 三种拓扑各验证 SVG 可正常渲染

B3: 混合检索 (Hybrid Retrieval)

目标: 融合 BM25 关键词检索, 提升型号精确匹配召回率

任务清单:

☐ 安装依赖: `pip install rank_bm25`

☐ 新增 `app/hybrid_retrieval.py` :

```
class HybridRetriever:
    def __init__(self, chroma_collection, documents):
        self.bm25 = BM25Okapi([doc.split() for doc in documents])
        self.chroma = chroma_collection

    def retrieve(self, query: str, k=5) -> list[str]:
        bm25_scores = self.bm25.get_scores(query.split())
        bm25_top = np.argsort(bm25_scores)[-k:][::-1]
        vector_results = self.chroma.query(query_texts=[query],
        n_results=k)
        return self._rrf_merge(bm25_top, vector_results, k=k)
```

☐ 集成到 `SearchNode`

☐ 评估: 对比混合检索 vs 纯向量检索的召回率

B4: 语义缓存层

目标：重复或语义近似的请求跳过 LLM 调用，直接返回缓存结果

任务清单：

- ☐ 新增 `app/semantic_cache.py` :
 - `get(query)` → 查询 ChromaDB 缓存集合， $\text{cosine} > 0.95$ 则命中
 - `set(query, result)` → 存入缓存集合
 - ☐ 集成到 `ParseNode` 前置
 - ☐ 响应 Header: `X-Cache: HIT/MISS`
-

B5：知识库扩展（29 → 50+ 条）

新增条目覆盖：

- ☐ 矽力杰 (Silergy) Buck/LDO: SY8240、SY8113B — 高效率、小封装
 - ☐ 圣邦微 (SGMICRO) : SGM2019 LDO — 低压差、低噪声
 - ☐ 南芯半导体 (Southchip) : SC8905 Boost — 高耐压、工业级
 - ☐ 稳压 IC 外围元件规则：电容 ESR 要求、电感饱和电流裕量
 - ☐ AEC-Q100 Grade 0/1/2 区别与选型建议
 - ☐ 车规供应链集中度风险量化规则
 - ☐ 格式沿用 `data/knowledge/engineering_knowledge.json`
-

B6：BOM 清单 Excel 导出

- ☐ 安装依赖: `pip install openpyxl`
 - ☐ 新增 `app/bom_exporter.py` :
 - Sheet 1: BOM 主表 (MPN、描述、Vin、Vout、Iout、等级、评分、风险等级、置信度)
 - Sheet 2: 风险详情 (每条风险条目展开)
 - Sheet 3: 证据链记录
 - 格式: HIGH 红底、MEDIUM 橙底、LOW 绿底
 - ☐ 新增端点 `/export/bom`, 返回 `.xlsx` binary
-

五、集成与联调要点

接口约定（前后端对齐）

端点	方法	请求	响应	负责人
/analyze/stream	POST	{query:string}	SSE text/event-stream	队长 + 队友B
/schematic/{topology}	GET	?Vin&Vout&Iout	SVG text	队友B
/export/bom	POST	SelectionReport JSON	.xlsx binary	队友B
/report/pdf/{session_id}	GET	path param	PDF binary	队友B
/agent/chat	POST	保持原有	JSON	队长

联调顺序

Step 1: 队友B 完成 B1 (SSE)	→ 队友A 对接 F2 (流式打字)
Step 2: 队长 完成 P1 (LangGraph)	→ 队友B 将 SSE 接口接入 LangGraph
Step 3: 队友B 完成 B2 (电路图)	→ 队友A 完成 F4 (SchematicPanel)
Step 4: 队长 完成 P3 (消融实验)	→ 更新 PPT 实验章节数据
Step 5: 三人完成各自模块	→ 端到端集成测试 (10 条用例全链路验证)

六、Demo 完成标准

演示时必须能展示以下完整流程：

- ☐ 在 Web 界面输入自然语言需求（展示 ≥ 3 种不同场景）
- ☐ 流式打字效果实时输出报告（无白屏等待）
- ☐ 对话下方展示选型结果卡片（含评分进度条、风险 Badge）
- ☐ 点击"查看电路图"弹出参数化应用电路 SVG（含计算值标注）
- ☐ 切换 Tab 查看风险热力矩阵
- ☐ 点击"导出 BOM"下载格式化 Excel
- ☐ 会话历史侧边栏显示多条历史记录，可切换回看
- ☐ PPT 实验章节有消融实验矩阵 + Baseline 对比表（数字说话）

七、文件变动索引

新增/修改文件（按负责人）：	
队长：	
app/agent_orchestrator.py	← LangGraph 重构（核心）
app/react_agent.py	← 接入 LangGraph 图
app/llm_client.py	← Function Calling 替换 JSON 解析
tests/eval_runner.py	← 消融实验参数 + 扩展用例
tests/test_cases_60.xlsx	← 新增 32 条用例

docs/eval_results_ablation.md ← 消融实验结果表格

队友A:

frontend/web/ ← Next.js 全新项目

frontend/web/app/layout.tsx ← 三栏布局

frontend/web/components/ ←

MessageBubble/ReportCard/SchematicPanel/RiskHeatmap

frontend/web/store/ ← Zustand session store

队友B:

app/main.py ← 新增4个端点

app/schematic_generator.py ← schemdraw 电路图 (新建)

app/hybrid_retrieval.py ← 混合检索 (新建)

app/semantic_cache.py ← 语义缓存 (新建)

app/bom_exporter.py ← Excel 导出 (新建)

data/knowledge/engineering_knowledge.json ← 扩展至50+条

requirements.txt ← 新增 schemdraw/rank_bm25/openpyxl